



SOICT

WEB MINING

COURSE: IT4868E

LEARNING EQUALITY CURRICULUM RECOMMENDATIONS

Group 19

Author

NGUYEN HAI DANG
NGUYEN TIEN DAT
PHAN MINH HOA
NGUYEN VO NGOC KHUE
NGUYEN HOANG QUAN

Mentor

DR. NGUYEN KIEM HIEU

December 2025

Contents

1. Problem Introduction	2
1.1 Background & Motivation	2
1.2 Problem Statement and Scope	2
1.3 Alignment with Web Mining Curriculum	3
2. Data Description	4
2.1 Dataset Overview	4
2.2 Data Format and Structure	5
2.3 Exploratory Data Analysis and Key Observations	6
2.4 Data Preprocessing and Preparation	13
2.4.1 Text normalization and cleaning	13
2.4.2 Language-aware segmentation	14
2.4.3 Text representation and feature-oriented construction	15
2.4.4 Data splitting and stratification	16
3. Methodology: Two-Stage Ensemble Reranking Pipeline	17
3.1 Stage 1 – Candidate Retrieval (Vector Space Model)	17
3.1.1 TF–IDF Representation	18
3.1.2 Efficient Similarity Search	18
3.2 Stage 2 – Learning-to-Rank Re-Ranker	19
3.2.1 Feature Vector Construction	19
3.2.2 Learning-to-Rank Objective	21
3.3 Ensemble of Multiple Models	22
3.4 Post-processing and Thresholding	23
4. Evaluation	24
4.1 Evaluation Protocol and Metrics	24
4.1.1 Data splits and cross-validation	24
4.1.2 Official metric: micro- F_2 score	24
4.1.3 LightGBM configuration and training objective	25
4.1.4 Complementary IR metrics	25
4.2 Offline Validation Results	26
4.3 Leaderboard / Test-set Performance	27
4.4 Ablation and Variant Analysis	27
4.4.1 Effect of candidate pool size (Stage 1 retrieval)	28
4.4.2 Backbone capacity and embedding weight ablation	28
4.4.3 Effect of feature groups in Stage 2	29
4.4.4 Ensemble depth and model diversity	29
4.4.5 Post-processing and thresholding	30
5. Conclusion	30

1. Problem Introduction

1.1 Background & Motivation

The global landscape of education has undergone a significant digital transformation in recent years, especially with the accelerated adoption of online and remote learning resources. However, a critical challenge remains in efficiently aligning the vast array of available educational content with specific curriculum objectives. This challenge is particularly acute in contexts where access to expert educators is limited – for example, in developing regions, refugee camps, or areas impacted by crises (such as pandemic-related school closures). In such settings, finding appropriate learning materials for a given topic or standard is a time-consuming, manual process that can take educators weeks or months for large content repositories.

Learning Equality, an open-source education technology initiative in partnership with the UN Refugee Agency (UNHCR) and The Learning Agency Lab, has identified curriculum alignment as a bottleneck in delivering quality education to underserved communities. Currently, matching educational content (videos, exercises, texts, etc.) to curriculum topics is largely done by human experts, requiring extensive domain knowledge and effort. Automating this alignment through machine learning would free up educators' time to focus on teaching and student interaction, rather than curation tasks. Moreover, for refugee learners and others in low-resource contexts, improved curriculum alignment directly impacts educational equity - ensuring that learners have access to relevant content mapped to their learning needs. Efficient alignment tools can enhance the quality and personalization of education for millions of learners worldwide.

1.2 Problem Statement and Scope

The Learning Equality – Curriculum Recommendations challenge on Kaggle (hosted Dec 2022-Mar 2023) [9] addresses this alignment problem by soliciting algorithms to automatically match educational content items to the appropriate topics in a curriculum tree. In this task, a topic represents a node in a curriculum hierarchy (for example, a learning objective like "Triangles and Polygons"), described by a title and possibly a detailed description, as well as its position in a taxonomy of subjects. A content item represents an educational resource (e.g. a video, reading, or exercise) with its own metadata (title, description, etc.). The goal is: given a curriculum topic, predict a ranked list of content items that are most relevant and aligned to that topic. Each topic may have multiple relevant content items, and the algorithm must retrieve and rank these from a large corpus of candidates.

Inputs: The input to the system is a topic from the curriculum, represented by textual fields - typically the topic's title, an optional descriptive text elaborating the topic's scope, and the hierarchical context (e.g. the titles of parent or ancestor topics in the curriculum tree which situate the topic). This hierarchical context is important, as it provides additional semantic cues (for instance, a topic titled "Cell Division" under a parent topic "Biology" should be interpreted differently than the same title under "Mathematics").

Outputs: The expected output is a ranked list of content recommendations for the given topic. That is, the system should return a list of content item IDs (from the content repository) ordered by predicted relevance to the query topic. Only the most relevant items should appear at the top of the list. The ranking is evaluated on how well these items align with the ground-truth curriculum

alignments provided in the dataset (i.e., which content items were labeled as relevant to each topic by experts).

Scale and Challenges: This problem involves a large-scale retrieval scenario with multilingual data across diverse K-12 subjects. The dataset contains 76972 topics with content associations and over 154,047 content items (see Data Description below), making efficient search and ranking essential. Topics and content are drawn from multiple languages (including English, Spanish, French, Swahili, Arabic, etc.), reflecting the global usage of Learning Equality’s Kolibriwe platform. This multilingual aspect introduces additional complexity: the system must match topics and content even when they are in different languages or when wording varies across languages, requiring language-agnostic semantic understanding. Moreover, the content items vary in format (videos, exercises with Q&A, text articles, etc.) and length, which means the recommendation system must be robust to different types of textual data.

In summary, the scope of the project is to develop a two-stage information retrieval pipeline capable of retrieving relevant content items for a given curriculum topic and then ranking those items by relevance. The solution must handle a retrieval set on the order of millions of items and output high-quality rankings under pragmatic constraints of time (inference efficiency) and space. Our solution achieved this through an ensemble of modern IR techniques, which we will describe in detail.

1.3 Alignment with Web Mining Curriculum

While the application domain is educational recommendation, this project is methodologically grounded in **Information Retrieval (IR)**, supplemented by concepts from **Content-Based Filtering**. This distinction aligns directly with the core topics of the Web Mining course:

1. **Information Retrieval Models:** Fundamentally, the task is modeled as a retrieval problem where we match complex queries to documents. Here, curriculum topics act as query objects, and content items act as the document collection. We apply classical IR techniques such as the Vector Space Model (VSM), representing text as high-dimensional vectors and utilizing cosine similarity for efficient candidate generation [1]. This aligns with the course’s foundational modules on ranking, relevance scoring, and indexing.
2. **Semantic Embeddings and NLP:** To transcend the limitations of keyword overlap, the solution integrates semantic text embeddings, vector representations learned by Transformer-based language models. We employ `bert-base-multilingual-uncased` (mBERT), `xlm-roberta-base` (XLM-R base), and `xlm-roberta-large` (XLM-R large) [3, 4, 5]. This reflects the course’s coverage of Natural Language Processing, moving from basic preprocessing (tokenization, normalization) to modern dense retrieval methods. The multilingual nature of the dataset further extends the course’s discussion on handling diverse linguistic topologies.
3. **Learning-to-Rank (LTR):** Instead of relying on a single heuristic metric, we implement a Learning-to-Rank framework to optimize the ranking function directly [2]. By training a gradient-boosted decision tree (LightGBM) on pairwise relevance features, the model learns to minimize ranking errors. This is a practical application of the course’s machine learning modules, demonstrating how to optimize ranking objectives (like NDCG) using supervised learning [6, 7, 8].

4. **Content-Based Filtering Concepts:** Although not a collaborative filtering system involving user interaction history, the project leverages principles from Content-Based Recommendation. We treat the curriculum topic as a "context profile" and the educational resources as "items." The feature engineering process, extracting textual, structural, and metadata similarities, mirrors the content-based filtering approach of matching item attributes to a target profile. This highlights the intersection where IR techniques serve as the engine for content-based recommendation logic.
5. **Evaluation Metrics:** The system is evaluated using standard IR metrics emphasized in the course, specifically Mean Reciprocal Rank (MRR) and Normalized Discounted Cumulative Gain (NDCG) [8]. These metrics are chosen to assess the quality of the ranked lists, ensuring that the most relevant educational content appears at the top of the recommendations, which is critical for user trust in search and recommendation interfaces.

Overall, this project integrates these concepts into a cohesive solution. By addressing a real-world problem of aligning educational resources, it provides practical experience with the tools and techniques discussed in Web Mining, particularly in building end-to-end retrieval and ranking pipelines for large-scale text data.

2. Data Description

In this section, We describe the Learning Equality – Curriculum Recommendations (LECR) dataset used in this project, its file structure, key statistical properties, and the preprocessing pipeline that prepares raw data for the two-stage retrieval and reranking system. The goal is to make explicit how the characteristics of the dataset influence the design of my models and features.

2.1 Dataset Overview

The LECR dataset is derived from Learning Equality’s Kolibrwe Studio platform, which hosts open educational resources and curriculum structures for K–12 learners worldwide. It consists of curriculum topics, learning content items, and expert-curated alignments between them. At a high level, the dataset can be summarized as follows:

- **Topics:** Nodes in curriculum trees that represent learning objectives such as “Triangles and Polygons” or “Photosynthesis”. Topics are organized hierarchically by subject, grade, unit, and sub-unit.
- **Content items:** Educational resources aligned to topics, including videos, exercises, documents, HTML5 apps, and audio materials. Content is drawn from open educational repositories.
- **Alignments:** Ground-truth relationships indicating which content items are relevant to which topics, labeled by curriculum experts.
- **Multilingual coverage:** Topics and content span multiple languages, reflecting Kolibri’s deployment in diverse regions.

Table 1 reports approximate dataset statistics for the competition version of the corpus.

Table 1: Learning Equality – Curriculum Recommendations dataset statistics

Component	Count (approx.)
Total curriculum topics	76,972
Total content items	154,047
Training alignments (topic–content pairs)	279,919
Languages represented	28

The combination of large scale, multilinguality, and many-to-many topic–content relationships makes this dataset a realistic but challenging benchmark for information retrieval (IR) and recommendation methods.

2.2 Data Format and Structure

The dataset is distributed as several CSV files. In this project We primarily use three core files:

1. `topics.csv`: curriculum topic metadata,
2. `content.csv`: learning resource metadata and text,
3. `correlations.csv`: ground-truth topic–content relationships.

Topics

Each row in `topics.csv` corresponds to a curriculum topic and contains:

- **id**: unique topic identifier (e.g., `t_12345`).
- **title**: short title summarizing the learning objective (e.g., “Triangles and Polygons”).
- **description**: longer textual description of what learners should know or be able to do. This field can be empty for some topics.
- **language**: ISO language code of the topic text (e.g., `en`, `es`, `fr`, `sw`).
- **parent_id**: identifier of the parent topic in the curriculum tree, or `NaN` for root nodes.
- **channel**: identifier of the curriculum or content provider (e.g., a national curriculum or NGO curriculum).
- **has_content_flag**: Boolean indicator of whether this topic has known aligned content in the training data.

The `parent_id` field defines a forest of topic trees: each channel corresponds to a separate curriculum, and topics form hierarchical paths such as

Grade 7 Mathematics → Geometry → Shapes → Triangles and Polygons.

This structure is useful both for constructing richer textual descriptions (by concatenating ancestor titles) and for understanding the granularity of topics.

Content items

Each row in `content.csv` corresponds to a learning resource and includes:

- **id**: unique content identifier (e.g., `c_67890`).
- **title**: title or heading of the resource.
- **description**: short description of the resource; can be missing for some items.
- **text**: raw textual body when available (e.g., exercise text, video subtitles, or document text). This field may be empty for purely multimedia resources.
- **kind**: resource type (e.g., `video`, `exercise`, `document`, `html5`, `audio`).
- **language**: language code of the resource.
- **license** and other metadata: copyright/license information and provider-specific attributes.

This rich representation allows both metadata-based matching (e.g., by kind or language) and full-text matching using IR techniques and transformer-based embeddings.

Ground-truth correlations

The `correlations.csv` file provides supervised labels in the form of expert-curated curriculum alignments. Each row associates a topic with one or more relevant content IDs:

- **topic_id**: identifier of a training topic.
- **content_ids**: space-separated list of content IDs that are considered correct recommendations for that topic.

These alignments serve three roles in this project:

1. as positive labels to train the learning-to-rank model,
2. as ground truth for validation metrics such as MRR and NDCG,
3. as a basis for constructing hard negatives and second-degree relationships (via similar training topics) during feature engineering.

2.3 Exploratory Data Analysis and Key Observations

Before designing retrieval and ranking models, We performed exploratory data analysis (EDA) to understand structural and statistical properties of the dataset. Several patterns directly influence modelling choices.

Topic hierarchy and coverage

Topics form curriculum trees with depths typically between 3 and 6 levels. Core STEM subjects such as Mathematics and Science exhibit dense, deep hierarchies with many specialized child topics, whereas some humanities and language curricula are shallower and less densely populated. Certain channels contain thousands of topics, while others contain only a few hundred. This non-uniform structure implies that:

- topics at deeper levels tend to be more specific and require more fine-grained matching,
- ancestor titles and channels carry useful contextual signals that should be injected into topic representations.

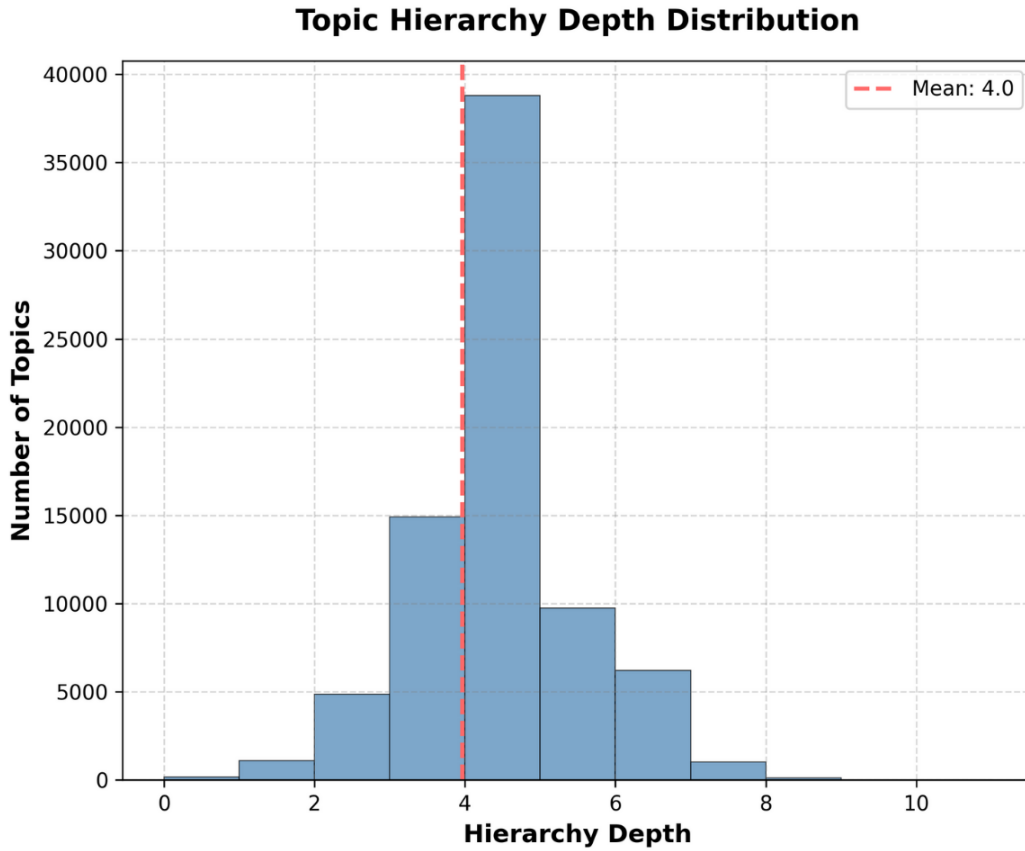


Figure 1: Topic hierarchy depth distribution.

Content availability and sparsity

Content availability is highly imbalanced across topics and languages. Some topics are aligned with hundreds of content items, while others have fewer than ten. Overall, only a tiny fraction of all possible topic–content pairs are positives: each topic typically has between 1 and 20 aligned items out of a candidate pool of hundreds of thousands. Formally, let

$$N_{\text{topics}} = |T|, \quad N_{\text{content}} = |C|, \quad N_{\text{pos}} = |\{(t, c) \mid c \text{ is aligned to } t\}|.$$

Then the proportion of positives among all possible pairs is

$$\pi_{\text{pos}} = \frac{N_{\text{pos}}}{N_{\text{topics}} \cdot N_{\text{content}}} \ll 10^{-4}.$$

This extreme class imbalance motivates a two-stage design: high-recall candidate retrieval followed by a focused learning-to-rank model on a small candidate subset.

Language distribution and multilingual challenges

English content is the most abundant (roughly 60% of all content items), followed by Spanish, French, and several other languages such as Swahili and Arabic. Topics and content are annotated with language codes, and the majority of true alignments occur within the same language. However, there are also multilingual channels and cross-lingual alignments (e.g., English topics aligned to content in another language). This leads to two practical decisions:

- perform retrieval separately within each language group to reduce the candidate space and avoid spurious matches,
- rely on multilingual transformer embeddings to handle rare cross-lingual matches when they are meaningful.

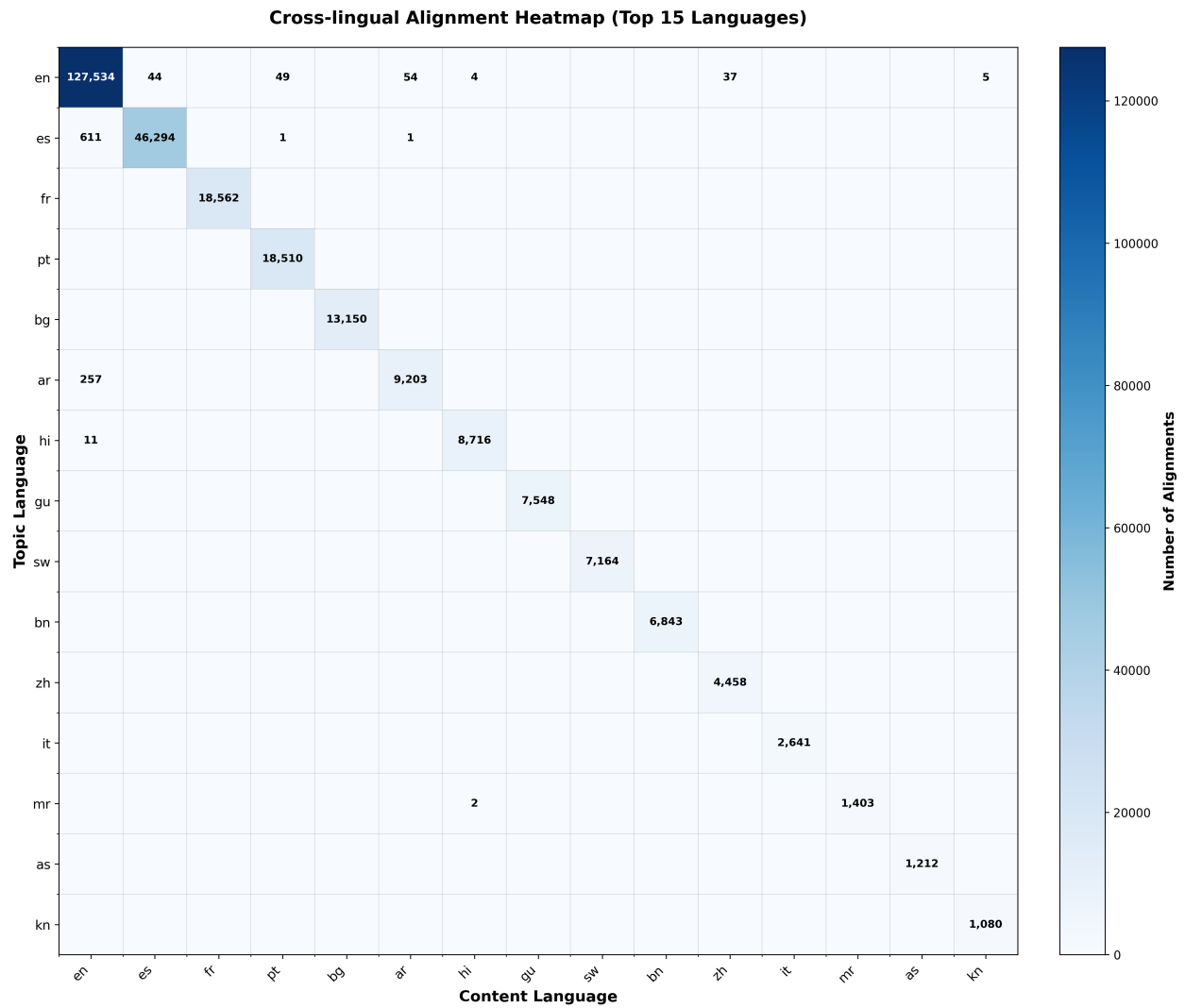


Figure 2: Heatmap of multilingual curriculum

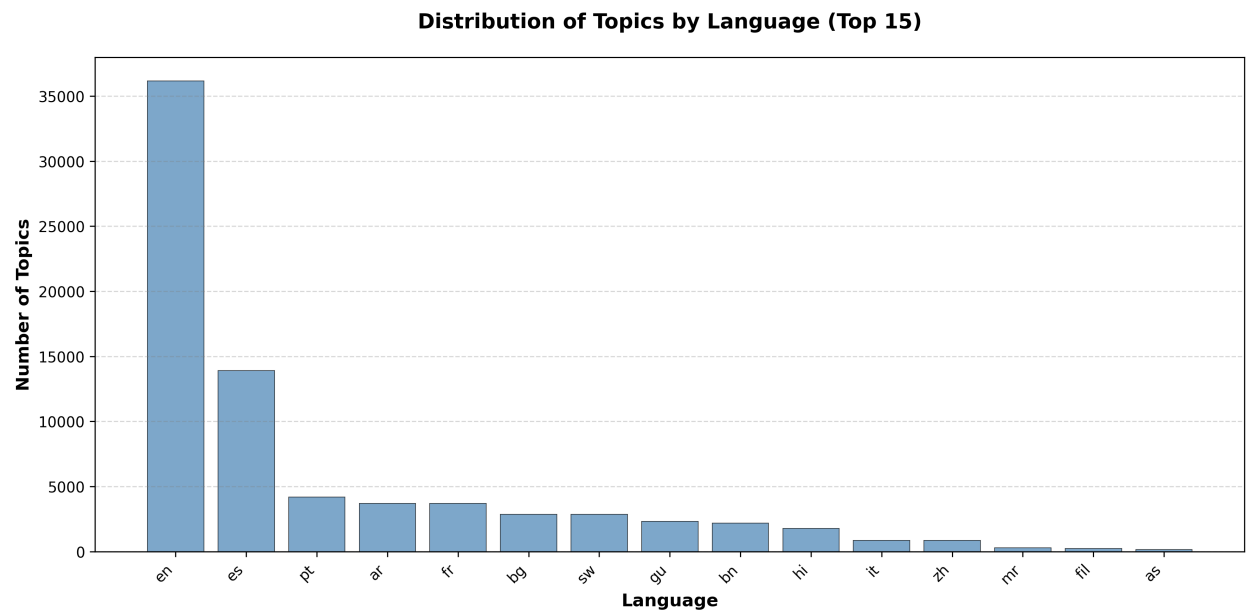


Figure 3: Distribution of topics and content items by language (top 15 languages).

Text length and density

Text fields show substantial variation in length:

- topic titles are short (a few words), while descriptions range from tens to several hundred words,
- content titles are typically short, but full text fields can range from a single sentence (for a simple exercise) to many paragraphs (for a transcript or long document).

This variation affects both sparse TF-IDF representations and dense embedding quality. Very short texts can produce noisy embeddings and sparse lexical matches; overly long texts may dilute key terms. In response, we:

- construct composite topic texts that include ancestor titles to enrich short descriptions,
- truncate extremely long content text to a fixed maximum length for embedding models while retaining full text for TF-IDF.

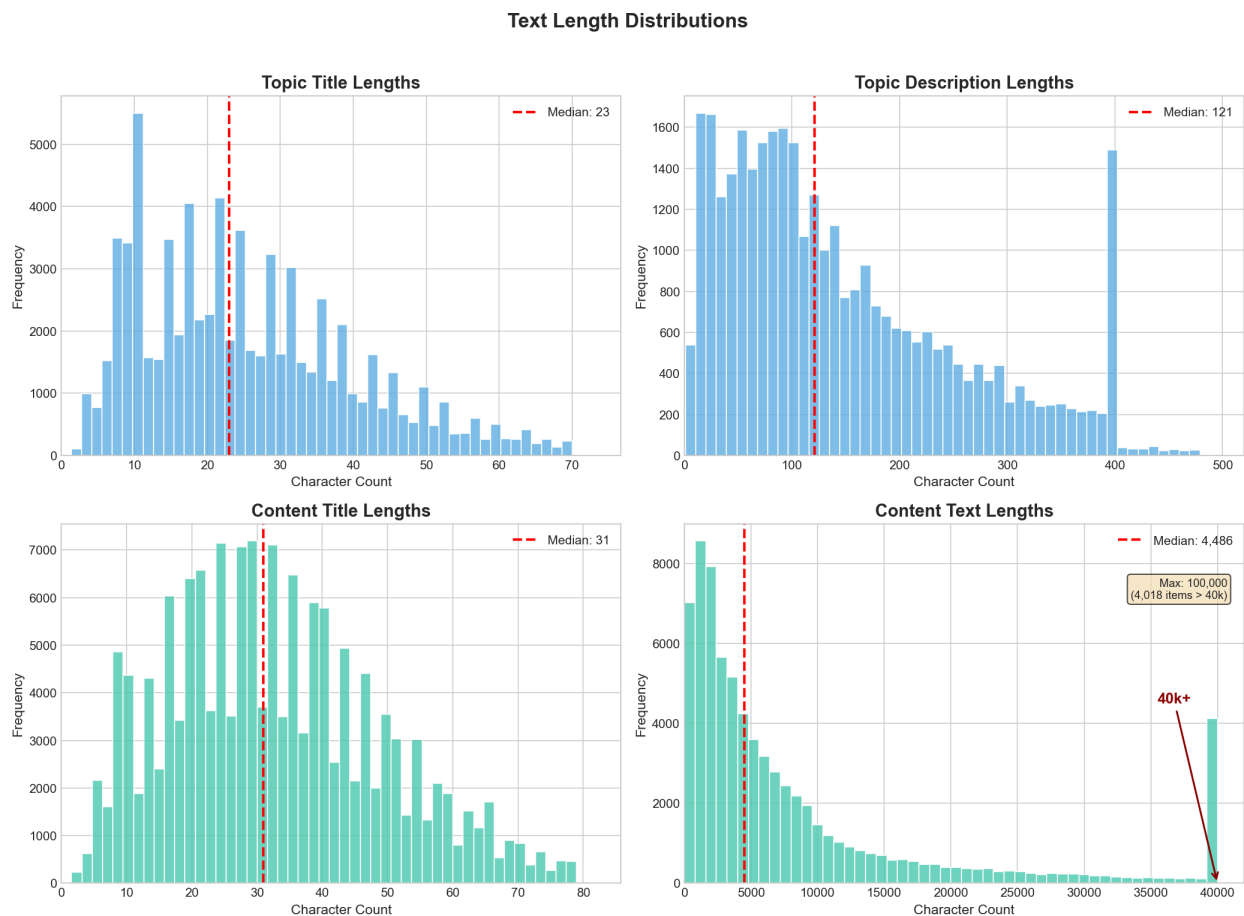


Figure 4: Text length distributions for topic and content fields.

Semantic overlap and ambiguity

EDA also reveals substantial semantic overlap between topics. For example, topics like “Fractions”, “Rational Numbers”, and “Operations with Fractions” share vocabulary but differ in scope and grade level. Content items can also be relevant to multiple topics at different depths. This reinforces the need for:

- semantic embeddings that capture nuanced meaning beyond exact word overlap,
- a learning-to-rank model that can disentangle fine-grained relevance within sets of very similar candidates.

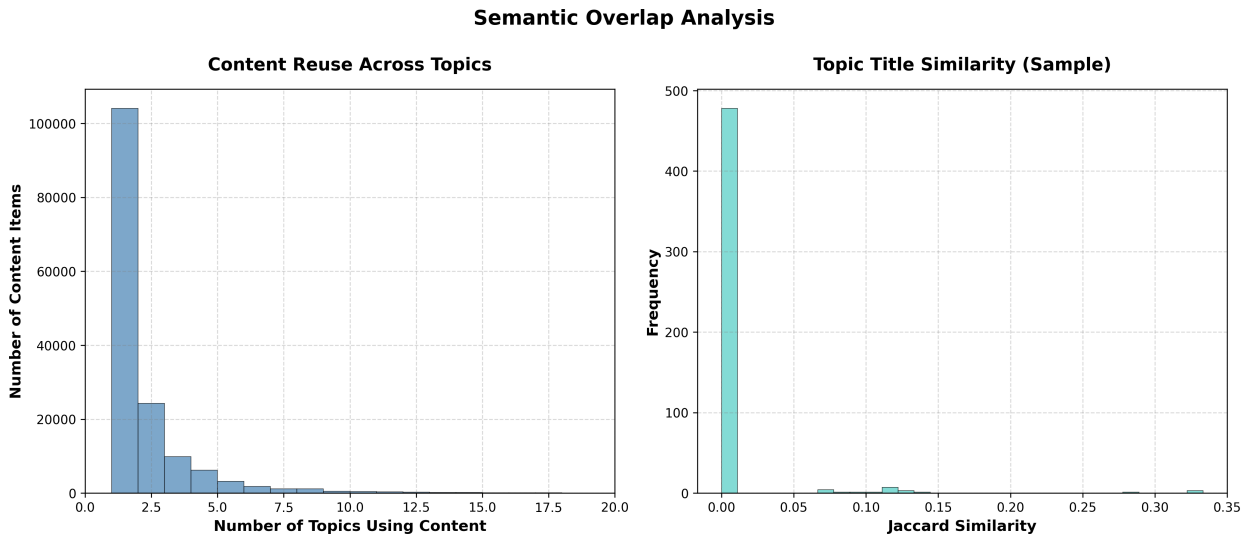


Figure 5: Semantic overlap analysis: Content reuse across multiple topics (left) and Jaccard similarity of topic titles within channels (right).

Content type distribution

The LECR dataset includes diverse content types (“kinds”), with varying pedagogical purposes. Figure 6 shows the distribution of content by type. Videos and exercises constitute the majority of resources, while documents, HTML5 interactive apps, and audio materials serve specialized purposes. This variety requires the recommendation system to match topics to content regardless of format, relying purely on textual metadata rather than format-specific features.

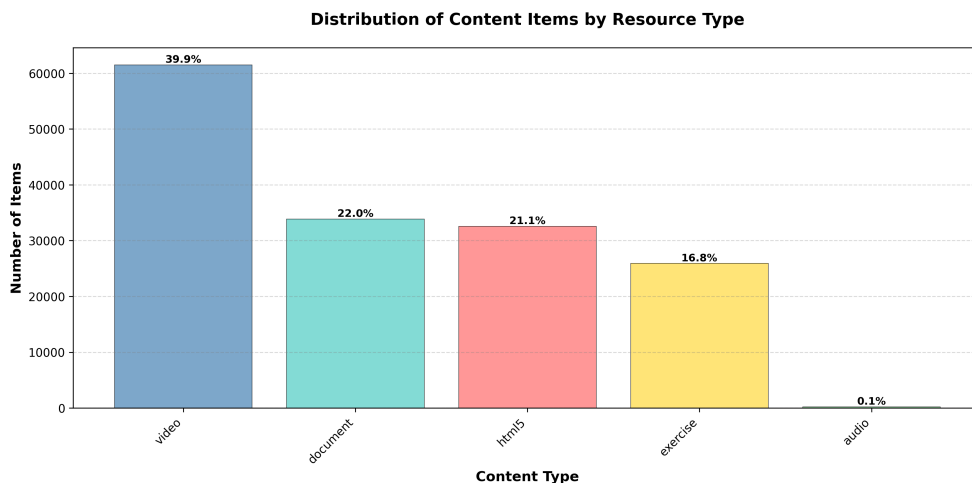


Figure 6: Distribution of content items by resource type (video, exercise, document, html5, audio).

Data quality and missing values

An important consideration for feature engineering is the presence of missing data. Figure 7 presents the proportion of missing values for each field in the topics and content datasets. While

core fields like “id” and “title” are nearly complete, description and text fields exhibit higher missingness rates. This motivates the composite text construction strategy described in Section 2.4, where we gracefully fall back to available fields when primary fields are absent.

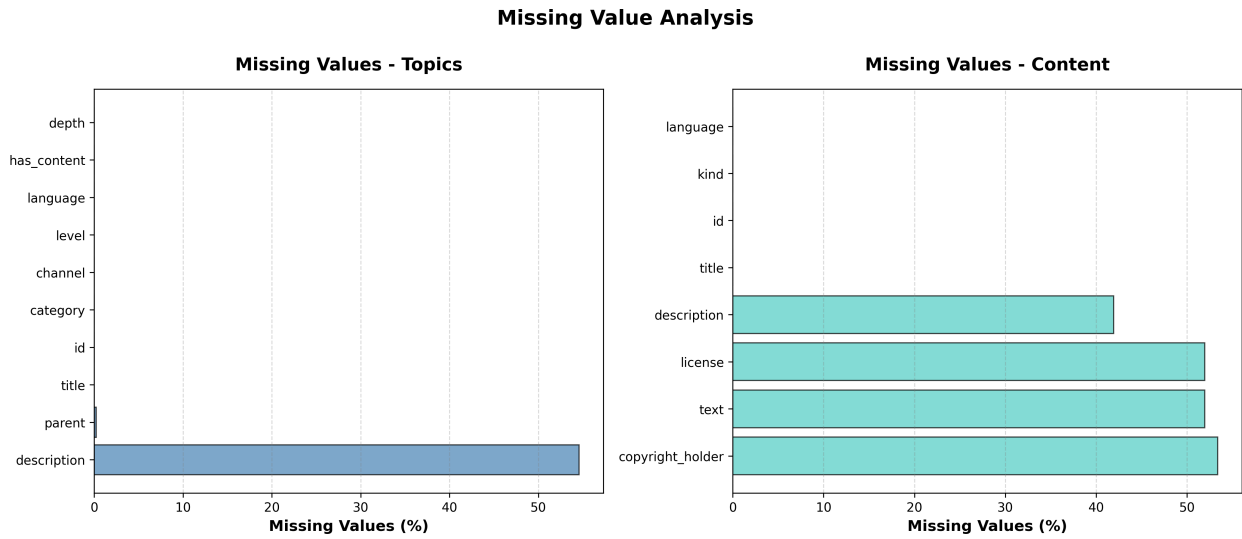


Figure 7: Missing value analysis for topics (left) and content (right) datasets.

Channel and curriculum distribution

The dataset encompasses multiple curriculum channels from different educational providers and national curricula. Figure 8 shows that channel sizes vary dramatically, some contain tens of thousands of topics while others have only a few hundred. The right panel shows total alignments per channel, which correlates with but is not identical to topic counts, since different curricula have varying coverage density. This non-uniform distribution motivates our cross-validation strategy using channel-level grouping to prevent curriculum-specific patterns from leaking across folds.

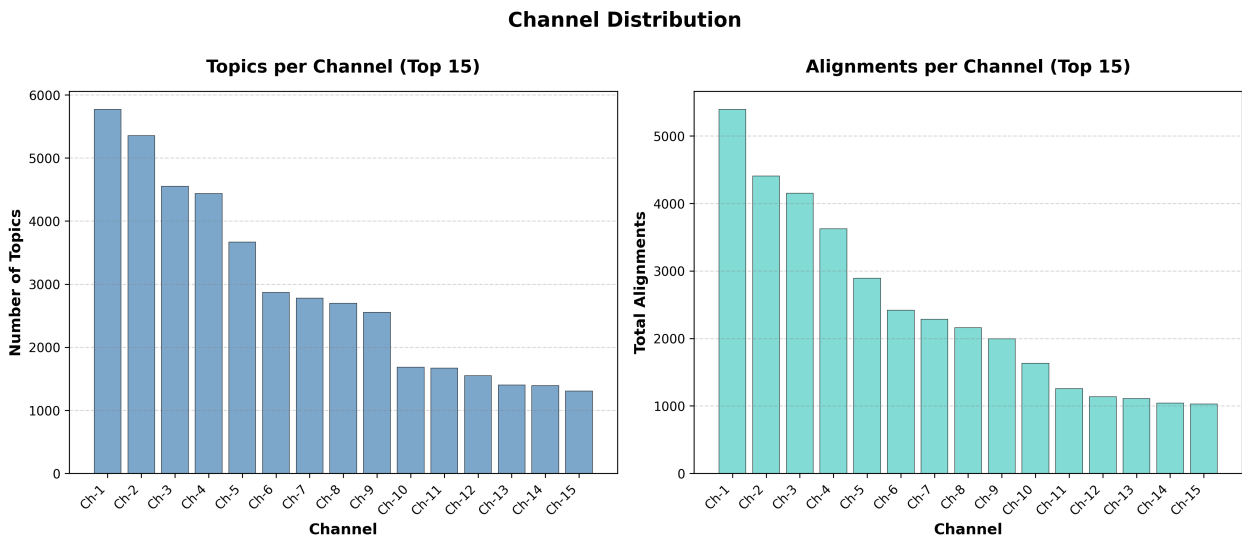


Figure 8: Channel distribution: Number of topics per channel (left) and total alignments per channel (right).

Content reuse patterns

Beyond the semantic overlap analysis, we also examine how content items are shared across topics. Figure 9 shows the distribution of topics each content item serves. The majority of content is aligned to only 1–3 topics, indicating relatively specific matching. However, a long tail of content items serve 10+ topics, representing either highly versatile foundational resources or broad instructional materials (e.g., introductory videos on general concepts). This asymmetric reuse pattern is another motivation for the learning-to-rank approach, which can learn context-specific relevance rather than treating all alignments uniformly.

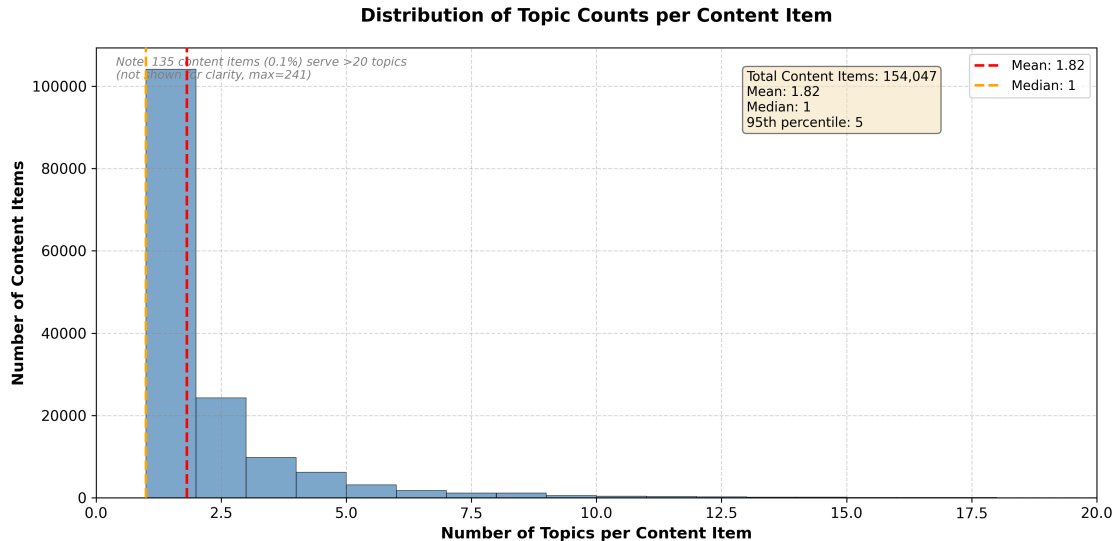


Figure 9: Distribution of topic counts per content item, showing content reuse patterns.

2.4 Data Preprocessing and Preparation

Based on the dataset characteristics observed above, We designed a preprocessing pipeline that standardizes and enriches textual representations while respecting multilingual and hierarchical structure. The main stages are: text normalization, language-based segmentation, feature-oriented text construction, and data splitting.

2.4.1 Text normalization and cleaning

Text preprocessing is applied to both topic and content text fields (titles, descriptions, and where available, full text). The steps are:

Lowercasing and Unicode normalization

All text is converted to lowercase (for scripts where case is meaningful) to reduce vocabulary size and improve matching. We then apply Unicode normalization (e.g., NFKD) to handle accented characters and compatibility characters consistently. This ensures that visually similar characters are mapped to a canonical form before tokenization.

Punctuation, markup, and special symbols

HTML tags and markup artifacts are stripped from the raw text. Punctuation is generally preserved because:

- sentence boundaries and list markers can help transformer tokenizers,
- mathematical notation and symbols (e.g., “+”, “=”, fractions) are pedagogically important.

For some auxiliary features (character n -grams), we remove control characters and normalize repeated punctuation but keep alphanumeric and basic symbol characters.

Whitespace normalization and tokenization

Multiple whitespace characters are collapsed to a single space. For classical IR features (TF-IDF), We apply a simple whitespace and punctuation-based tokenization to obtain word tokens, which are later passed to the vectorizer. For neural models, tokenization is handled by the underlying SentencePiece/WordPiece tokenizers, so We only ensure that the text is clean and normalized before model input.

Handling missing and redundant text

Many topics and content items have missing descriptions or highly redundant title/description pairs. To handle this:

- if a description is missing, we use only the title (and ancestor titles for topics),
- when the title and description are nearly identical, we replace the redundant description with either truncated full text (up to a fixed character limit, such as 256 characters) or leave it empty, to avoid repeating the same phrase twice,
- for content with available full text (e.g., transcripts), we construct a composite text field by concatenating title and an excerpt of the full text, using description only when it provides additional information.

This consolidation ensures that each field contributes complementary information rather than duplicating the same string.

2.4.2 Language-aware segmentation

Given the strong language-specific patterns observed in EDA, the pipeline processes data language by language:

- We use the dataset’s language column as the primary signal (no extra language detection is required for this competition).
- For each language ℓ , we create subsets $T^{(\ell)}$ and $C^{(\ell)}$ of topics and content items such that their language field equals ℓ .

- Candidate retrieval and TF–IDF vectorization are performed independently within each language group, which reduces the size of the sparse matrices and naturally enforces language-consistent matching.

This design has two benefits: it improves computational efficiency and allows the TF–IDF vocabularies and thresholds to adapt to language-specific morphology and character distributions.

2.4.3 Text representation and feature-oriented construction

After cleaning and segmenting the data, we build two complementary representations of text: sparse TF–IDF vectors and dense semantic embeddings.

Composite text fields

For topics, we construct a composite text string by concatenating ancestor titles (up to a fixed depth), the topic title, and the topic description:

$$\text{text}_{\text{topic}} = \text{ancestor_titles} || \text{title} || \text{description},$$

where “||” denotes string concatenation with sentence delimiters. For content, we concatenate title and either description or a truncated full text:

$$\text{text}_{\text{content}} = \text{title} || \text{description/full_text}_{\leq L}.$$

These composite fields feed both TF–IDF and embedding models, ensuring that they see comparable context.

TF–IDF character n -grams

To capture robust lexical patterns across languages, we employ a TF–IDF vectorizer over character 4-grams on concatenated topic and content composite texts. Character 4-grams are defined by sliding a window of length $n = 4$ over the text, e.g.,

$$\text{“learning”} \rightarrow \{\text{“lear”}, \text{“earn”}, \text{“arni”}, \text{“rnin”}, \text{“ning”}\}.$$

This representation is resilient to spelling variations, inflectional morphology, and tokenization issues.

For each term t (a 4-gram) and document d , the TF–IDF weight is:

$$w_{t,d} = \text{tf}(t, d) \cdot \log\left(\frac{N}{\text{df}(t)}\right),$$

where $\text{tf}(t, d)$ is the frequency of t in d , N is the total number of documents in the corpus, and $\text{df}(t)$ is the number of documents containing t . After weighting, we apply ℓ_2 normalization to each vector:

$$\hat{\mathbf{v}}_d = \frac{\mathbf{v}_d}{\|\mathbf{v}_d\|_2}.$$

To reduce noise and vocabulary size, we use a minimum document frequency threshold (e.g., $\text{min_df} = 2$), discarding 4-grams that appear in only a single document. The resulting vocabulary

typically consists of hundreds of thousands of cross-document 4-grams, which is manageable in sparse matrix form.

Dense semantic embeddings

In addition to sparse vectors, we generate dense semantic embeddings using pre-trained multi-lingual transformer models (detailed later in the Methodology section). For each composite text, a model maps the text to a fixed-dimensional vector:

$$f_\theta : \text{text} \mapsto \mathbf{e} \in \mathbb{R}^d,$$

where f_θ denotes a transformer encoder and d is the embedding dimension (e.g., $d = 384$ or 768). Embeddings are ℓ_2 -normalized so that cosine similarity between two texts reduces to a dot product between unit vectors:

$$\cos(\mathbf{e}_1, \mathbf{e}_2) = \frac{\mathbf{e}_1^\top \mathbf{e}_2}{\|\mathbf{e}_1\|_2 \|\mathbf{e}_2\|_2}.$$

These embeddings are later combined into features such as cosine similarity or cosine distance:

$$\text{vec_dist}(t, c) = 1 - \cos(\mathbf{e}_t, \mathbf{e}_c),$$

which serve as semantic matching signals for the learning-to-rank model.

Auxiliary statistical features

At preprocessing time, we also compute simple scalar features describing each text field:

- character counts for topic title, topic description, content title, content description, and content full text,
- counts of retrieved candidates per topic in the first-stage TF-IDF retrieval,
- language, channel, and kind categorical indicators.

These features are inexpensive to compute yet provide the ranking model with useful hints about specificity, granularity, and resource type.

2.4.4 Data splitting and stratification

Finally, we partition the labeled data into training and validation sets for supervised learning and model selection. Since topics, not individual pairs, are the natural units for recommendation, we split over topics rather than over topic-content pairs:

- We sample a subset of training topics as a validation set, ensuring that all their aligned content items are held out together.
- The remaining topics (and their associated alignments) form the training set.
- When necessary, we stratify splits by language and channel so that the validation set covers multiple curricula and languages.

This topic-level split mimics the competition’s cold-start setting: at validation and test time, models must generalize to topics that were unseen during training, using only their text and derived features. It also avoids leaking the same topic text into both training and validation, which would artificially inflate performance estimates.

Together, these data description and preprocessing steps define the foundation on which the two-stage ensemble reranking pipeline is built in later sections.

3. Methodology: Two-Stage Ensemble Reranking Pipeline

In this section, we describe the end-to-end approach that we adopt to solve the curriculum recommendation problem. Our method follows a classical two-stage Information Retrieval (IR) pipeline: a first-stage retriever that efficiently generates a small set of candidate content items for each topic, and a second-stage reranker that reorders these candidates using richer features and a learning-to-rank (LTR) model. The overall architecture is illustrated in Figure 10.

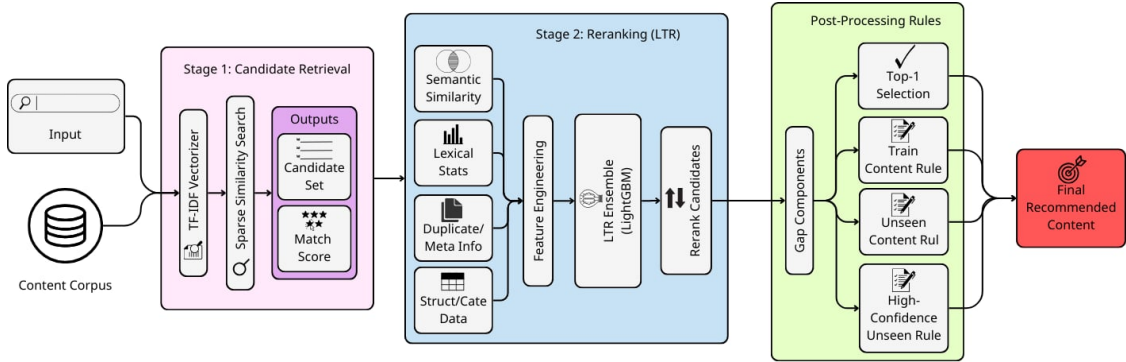


Figure 10: Two-stage retrieval and re-ranking architecture

In Stage 1, topics are encoded into sparse TF–IDF vectors and used to retrieve a small set of candidate content items from the full corpus via fast vector-space similarity. In Stage 2, we compute dense semantic and structural features for each topic–content pair and feed them into a gradient-boosted learning-to-rank model that outputs a refined relevance score. The final output for each topic is a ranked list of content items.

We denote by $\mathcal{T}$ the set of topics and by $\mathcal{C}$ the set of content items. Each topic $t \in \mathcal{T}$ and content item $c \in \mathcal{C}$ is associated with a textual representation (e.g., title and description), which we denote by $\text{text}(t)$ and $\text{text}(c)$ respectively. Our goal is to learn a scoring function

$$f : \mathcal{T} \times \mathcal{C} \rightarrow \mathbb{R},$$

such that, for each topic t , the content items c with higher scores $f(t, c)$ are more relevant to t .

3.1 Stage 1 – Candidate Retrieval (Vector Space Model)

Stage 1 focuses on high-recall retrieval: we want to quickly identify a relatively small set of candidate contents that are likely to be relevant for each topic while scanning a corpus of $|\mathcal{C}| \approx 10^6$ items. To achieve this, we adopt the classical Vector Space Model with TF–IDF representations and cosine similarity.

3.1.1 TF-IDF Representation

Let $\mathcal{V}$ denote the vocabulary built from the union of all topic and content texts. We map each text to a sparse TF-IDF vector in $\mathbb{R}^{|\mathcal{V}|}$:

$$: \text{Text} \rightarrow \mathbb{R}^{|\mathcal{V}|}, \quad \mathbf{x}_t = \varphi(\text{text}(t)), \quad \mathbf{x}_c = \varphi(\text{text}(c)).$$

For a term $w \in \mathcal{V}$, the TF-IDF weight in a document d (topic or content) is given by

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \text{idf}(w) = \frac{\text{freq}(w, d)}{\sum_{w' \in \mathcal{V}} \text{freq}(w', d)} \cdot \log\left(\frac{N}{\text{df}(w)}\right),$$

where N is the total number of documents (topics + contents), $\text{freq}(w, d)$ is the frequency of term w in d , and $\text{df}(w)$ is the number of documents containing w .

Given the TF-IDF vectors $\mathbf{x}_t$ and $\mathbf{x}_c$, we compute cosine similarity as

$$\text{sim}_{\text{tfidf}}(t, c) = \cos(\mathbf{x}_t, \mathbf{x}_c) = \frac{\mathbf{x}_t^\top \mathbf{x}_c}{\|\mathbf{x}_t\|_2 \|\mathbf{x}_c\|_2}.$$

3.1.2 Efficient Similarity Search

Naively computing $\text{sim}_{\text{tfidf}}(t, c)$ for every pair (t, c) would be prohibitively expensive:

$$\mathcal{O}(|\mathcal{T}| \cdot |\mathcal{C}| \cdot d),$$

where $d = |\mathcal{V}|$ is the TF-IDF dimension. Instead, we exploit the sparsity of TF-IDF vectors and use sparse matrix multiplication.

Let $T \in \mathbb{R}^{|\mathcal{T}| \times d}$ be the matrix whose i -th row is the TF-IDF vector of topic t_i , and let $C \in \mathbb{R}^{|\mathcal{C}| \times d}$ be the matrix whose j -th row is the TF-IDF vector of content item c_j . If all rows are L2-normalized, the cosine similarity matrix S can be written as

$$S = TC^\top \in \mathbb{R}^{|\mathcal{T}| \times |\mathcal{C}|}, \quad S_{ij} = \text{sim}_{\text{tfidf}}(t_i, c_j).$$

We never materialize the full dense matrix S . Instead, we rely on a routine such as `awesome_cossim_topn`, which efficiently computes, for each row i , only the top- N entries above a threshold τ :

$$\mathcal{C}_N(t_i) = \{c_j \in \mathcal{C} \mid S_{ij} \text{ is among the top } N \text{ scores and } S_{ij} \geq \tau\}.$$

In our implementation, we set $N = 20$ and $\tau = 0.01$. Thus, Stage 1 returns for each topic t a candidate set

$$\mathcal{C}_N(t) \subset \mathcal{C}, \quad |\mathcal{C}_N(t)| \leq 20,$$

together with their TF-IDF similarity scores $\text{sim}_{\text{tfidf}}(t, c)$, which we store as the `feature match_score`.

Outcome of Stage 1

Stage 1 is designed for high recall: the candidate set $\mathcal{C}_N(t)$ should contain almost all truly relevant content items for t . Precision at this stage can be relatively low, because Stage 2 will re-rank and filter candidates. Empirically, the TF-IDF retrieval alone achieves a strong baseline (around 70%

NDCG@20 on validation), indicating that the majority of correct matches are indeed captured in $\mathcal{C}_N(t)$, although their rank within the top- N list is suboptimal.

3.2 Stage 2 – Learning-to-Rank Re-Ranker

Stage 2 refines the ranking of the candidates returned by Stage 1. For each topic t and candidate content $c \in \mathcal{C}_N(t)$, we construct a feature vector $\mathbf{f}(t, c) \in \mathbb{R}^m$ and train a learning-to-rank model g that outputs a real-valued relevance score

$$s(t, c) = g(\mathbf{f}(t, c)),$$

such that, for a fixed topic t , relevant contents obtain higher scores than non-relevant ones.

3.2.1 Feature Vector Construction

The feature vector $\mathbf{f}(t, c)$ aggregates information from multiple sources: sparse lexical similarity, dense semantic embeddings, duplicate detection signals, training-set metadata, and structural cues. We organize the features into seven conceptually distinct groups:

(1) Semantic similarity features.

These capture deep semantic alignment between topic and content using pre-trained multilingual transformer embeddings. For the main track, we employ three backbones: bert-base-multilingual-uncased (mBERT-base, 768-dim), xlm-roberta-base (XLM-R base, 768-dim), and xlm-roberta-large (XLM-R large, 1024-dim). We compute weighted concatenated embeddings with weights $(w_1, w_2, w_3) = (0.3, 0.3, 0.4)$:

$$\mathbf{e} = [w_1 \mathbf{e}_{\text{mBERT}}, w_2 \mathbf{e}_{\text{XLM-R-base}}, w_3 \mathbf{e}_{\text{XLM-R-large}}] \in \mathbb{R}^{2560}.$$

For each pair (t, c) , we compute the cosine distance

$$\text{vec_dist}(t, c) = 1 - \cos(\mathbf{e}_t, \mathbf{e}_c),$$

where lower values indicate higher semantic similarity. To contextualize each candidate within its retrieval set, we also compute aggregate statistics:

$$\text{vec_dist_min}(t) = \min_{c' \in \mathcal{C}_N(t)} \text{vec_dist}(t, c'), \quad \text{vec_dist_max}(t) = \max_{c' \in \mathcal{C}_N(t)} \text{vec_dist}(t, c').$$

These features allow the ranker to distinguish whether a candidate is among the closest or farthest semantic matches for topic t . For the efficiency track, we use a single paraphrase-multilingual-MiniLM-L12-v2 encoder with 384-dim embeddings.

(2) Lexical / TF-IDF features.

These include the Stage 1 retrieval score `match_score` (character-level 4-gram TF-IDF cosine similarity) and, optionally, word-level TF-IDF overlap (`match_score2`). Aggregate statistics such as `match_score_max` and `match_score_min` provide context for each candidate’s lexical strength relative to the retrieval set.

(3) Duplicate detection features.

We identify near-duplicate topic–content pairs by matching the concatenated title and description of a test-set topic against the training set. For each pair (t, c) , we compute:

- `dup_count`: number of times this exact (title, description) pair appears in training data for content c .
- `tdup_count`: count based on title-only match.
- `pdup_count`: count based on the parent topic’s title.

We also include mean aggregates (e.g. `dup_count_mean`, `tdup_count_mean`, `pdup_count_mean`) across all candidates for topic t . High duplicate counts strongly indicate that the pair is a known alignment from the training curriculum.

(4) Second-degree matching.

For each test-set topic t , we find its nearest neighbor t_{train} in the training set (measured by embedding distance). If content c is aligned to t_{train} , we set a binary flag `second_degree` to true. This transitive matching signal helps the model generalize to new topics that are semantically close to training topics.

(5) Training-set metadata.

We include binary and continuous features that encode whether content c appears in the training set:

- `content_is_train`: binary indicator (true if c was seen during training).
- `content_max_train_score`: the maximum embedding distance between c and any training-set topic (defined only if `content_is_train` is true).
- `topic_max_train_score`: the embedding distance between test-set topic t and its nearest training-set neighbor.

These features allow the ranker to distinguish between known content (which can be recommended more confidently) and unseen content (which requires higher semantic similarity to be included).

(6) Structural features.

We encode curriculum-specific structural cues:

- `same_chapter`: binary flag indicating whether topic t and content c have the same chapter number (extracted from numeric prefixes in titles).
- `starts_same`: binary flag indicating whether the first word of the topic title matches the first word of the content title.

These features exploit the hierarchical structure of educational curricula, where content is often organized by chapter or section.

(7) Length and basic statistics.

We include character-level length features:

$$\begin{aligned} &\text{len_topic_title}(t), \text{ len_topic_description}(t), \\ &\text{len_content_title}(c), \text{ len_content_description}(c), \\ &\text{len_content_text}(c). \end{aligned}$$

These help the model detect extremely short or overly long texts, which may correlate with noisy or ambiguous matches. We also include categorical features such as `topic_language`, `topic_category`, `topic_channel`, `content_kind`, and `topic_level` to capture metadata-level compatibility.

Let $\mathbf{f}(t, c) \in \mathbb{R}^m$ denote the concatenation of all these features for a given pair. The total feature count m varies slightly between configurations: the main-track full-feature model uses approximately 30 features, while the efficiency track uses a reduced set of 25 features.

3.2.2 Learning-to-Rank Objective

For each topic t in the training set, let $\mathcal{C}^+(t)$ be the set of content items aligned to t in the ground-truth correlations file, and $\mathcal{C}^-(t)$ be the set of retrieved candidates that are not aligned. We assign binary relevance labels

$$y_{t,c} = \begin{cases} 1, & \text{if } c \in \mathcal{C}^+(t), \\ 0, & \text{if } c \in \mathcal{C}^-(t). \end{cases}$$

We train a gradient-boosted decision tree model (LightGBM) with a binary classification objective and AUC as the optimization metric. The model learns a scoring function $g(\mathbf{f}(t, c); \theta)$ parameterized by θ . The training objective is the binary cross-entropy loss:

$$\mathcal{L}(\theta) = - \sum_{(t,c)} \left[y_{t,c} \log \sigma(s_{t,c}) + (1 - y_{t,c}) \log(1 - \sigma(s_{t,c})) \right],$$

where $s_{t,c} = g(\mathbf{f}(t, c); \theta)$ is the model’s raw output and $\sigma(\cdot)$ is the sigmoid function. The AUC metric guides early stopping and model selection during training.

For final evaluation and model selection, we use the micro-averaged F_2 score with $\beta = 2$, which weights recall more heavily than precision:

$$F_2 = \frac{5PR}{4P + R},$$

where P and R are micro-averaged precision and recall across all topics. This metric aligns with the competition’s emphasis on comprehensive curriculum coverage.

For the main-track models, we enforce a monotonicity constraint on `vec_dist`: increasing semantic distance should monotonically decrease the predicted alignment probability. This regularization prevents the model from learning spurious correlations and improves generalization to out-of-distribution topics. We also use adaptive histogram binning (896 bins for `vec_dist`) to preserve fine-grained distance information.

At inference time, for each topic t we compute

$$s(t, c) = g(\mathbf{f}(t, c))$$

for all $c \in \mathcal{C}_N(t)$, and sort candidates in descending order of $s(t, c)$.

3.3 Ensemble of Multiple Models

In practice, we find that using multiple complementary models improves robustness. Our final system can be viewed as an ensemble at two levels: embedding-level and LTR-level.

Embedding-level ensemble.

We employ several multilingual transformer encoders to compute semantic similarities. The primary models used are:

- bert-base-multilingual-uncased (mBERT-base, 768-dim)
- xlm-roberta-base (XLM-R base, 768-dim)
- xlm-roberta-large (XLM-R large, 1024-dim, with a task-specific top layer)

For the main track, we use a weighted concatenation with weights $(w_1, w_2, w_3) = (0.3, 0.3, 0.4)$:

$$\mathbf{e} = [w_1 \mathbf{e}_{\text{mBERT}}, w_2 \mathbf{e}_{\text{XLM-R-base}}, w_3 \mathbf{e}_{\text{XLM-R-large}}] \in \mathbb{R}^{2560}.$$

This ensemble helps stabilize semantic matching across languages: if multiple encoders agree that a topic and a content item are semantically close, that pair is likely to be genuinely relevant. For the efficiency track, we use a single paraphrase-multilingual-MiniLM-L12-v2 encoder with 384-dim embeddings.

LTR-level ensemble.

We further ensemble multiple LightGBM ranking models. Concretely, we train models on different folds and with slightly different feature subsets. The final ensemble consists of 8 models:

- 4 models trained with the full feature set (including aggregate features like `vec_dist_max`, `match_score_min`)
- 4 models trained with a reduced feature set (excluding aggregate features to reduce overfitting)

Let $g_1, \dots, g_8$ denote the trained LTR models. At inference, we aggregate their predictions as

$$s_{\text{ltr}}(t, c) = \frac{1}{8} \sum_{m=1}^8 g_m(\mathbf{f}(t, c)).$$

This averaging reduces variance and leads to more stable rankings, especially on difficult topics. The diversity introduced by different feature subsets and fold splits ensures that the ensemble captures complementary failure modes.

3.4 Post-processing and Thresholding

After Stage 2, we obtain a scored list of candidates for each topic. We then apply a lightweight post-processing step to improve robustness and handle edge cases, particularly for content items not seen during training.

For each topic t , we define the *score gap* for a candidate c as

$$\text{gap}(c) = \max_{c' \in \mathcal{C}_N(t)} s(t, c') - s(t, c),$$

where $\mathcal{C}_N(t)$ is the candidate set for topic t . This measures how far below the top-ranked candidate a given item is.

We apply the following four inclusion rules to determine the final recommendation set:

1. **Top-1 guarantee.** Always include the highest-scoring candidate $c_{(1)}$ for each topic.
2. **Training-set content with small gap.** For content c seen during training (`content_is_train = true`), include c if

$$\text{gap}(c) < 0.25 \quad \text{and} \quad s(t, c) > 0.05.$$

This rule allows multiple known-good content items to be recommended if they are close in score to the top candidate.

3. **Unseen content with content-centric constraint.** For content c not seen during training, we additionally compute a *content-centric distance*:

$$\text{content_dist}(c) = \text{gap}(c) + \left(\max_{t'} s(t', c) - s(t, c) \right),$$

where the second term measures whether c has a better match with some other topic t' . We include c if it is the best match for that content across all topics and

$$\text{content_dist}(c) < 0.45.$$

This prevents over-assigning unseen content to many weakly related topics.

4. **High-confidence unseen content.** For unseen content with extremely high confidence, include c if

$$\text{gap}(c) < 0.05,$$

regardless of the content-centric constraint. This rule captures clear semantic matches for new content.

These post-processing steps are intentionally simple; the heavy lifting is done by the two-stage retrieval and LTR ensemble. Nevertheless, the score-gap and content-centric rules help balance precision and recall by avoiding very low-scoring content while still returning a diverse set of relevant items for each topic. In a real deployment, we could additionally enforce constraints such as avoiding duplicated or near-duplicate content, ensuring language consistency, or respecting grade-level boundaries. In the competition setting, metadata was limited, so we primarily relied on the rules above and de-duplication by content ID.

In summary, our methodology combines a fast, high-recall TF-IDF retriever with a powerful learning-to-rank ensemble that exploits multilingual transformer embeddings and engineered features. This design allows us to scale to millions of content items while maintaining high recommendation quality for a wide variety of curriculum topics.

4. Evaluation

This section evaluates the proposed two-stage ensemble reranking pipeline along two main axes: (i) ranking quality under the official competition metric, and (ii) additional Information Retrieval (IR) metrics that better expose the behaviour of the top of the list.

4.1 Evaluation Protocol and Metrics

4.1.1 Data splits and cross-validation

All models are trained and validated on the Learning Equality Curriculum Recommendations (LECR) training data. The evaluation protocol follows a group-aware cross-validation strategy to prevent data leakage across curriculum channels:

- **Group-aware cross-validation:** A 4-fold GroupKfold is applied with `topic_id` as the grouping key. All topics and their aligned content from the same curriculum are kept in the same fold, ensuring that no topic leaks into its validation fold. For the diverse ensemble variant, an alternative hash-based fold assignment is used: the last character of each `topic_id` is mapped through a dictionary `{"a": 10, "b": 11, ..., "f": 15}` and taken modulo 4 to assign fold membership. This diversifies the fold splits and reduces correlation between models in the ensemble.
- **Validation protocol:** For each fold, models are trained on 3/4 of the data and evaluated on the remaining 1/4. Metrics are reported as the average across folds. The final model is an ensemble trained on all folds and evaluated on the held-out test set.

4.1.2 Official metric: micro- F_2 score

The competition evaluates submissions with a micro-averaged F_β score with $\beta = 2$, i.e. the F_2 score. This metric prioritizes recall over precision, which is appropriate for curriculum alignment where missing relevant resources is more costly than including a few redundant ones.

For a given topic q , let TP_q , FP_q , and FN_q denote true positives, false positives, and false negatives among the predicted content items. Precision and recall are

$$P_q = \frac{TP_q}{TP_q + FP_q}, \quad R_q = \frac{TP_q}{TP_q + FN_q}. \quad (1)$$

The F_β score for topic q is

$$F_\beta(q) = \frac{(1 + \beta^2) P_q R_q}{\beta^2 P_q + R_q}, \quad \beta = 2. \quad (2)$$

The *micro* F_2 over all topics Q is obtained by first summing TP, FP, and FN across all topics, then computing micro-averaged precision and recall:

$$P_{\text{micro}} = \frac{\sum_{q \in Q} \text{TP}_q}{\sum_{q \in Q} (\text{TP}_q + \text{FP}_q)}, \quad R_{\text{micro}} = \frac{\sum_{q \in Q} \text{TP}_q}{\sum_{q \in Q} (\text{TP}_q + \text{FN}_q)}, \quad (3)$$

and finally

$$F_2^{\text{micro}} = \frac{(1 + 2^2) P_{\text{micro}} R_{\text{micro}}}{2^2 P_{\text{micro}} + R_{\text{micro}}} = \frac{5 P_{\text{micro}} R_{\text{micro}}}{4 P_{\text{micro}} + R_{\text{micro}}}. \quad (4)$$

The weight $\beta = 2$ means recall contributes four times as much as precision to the final score, reflecting the domain requirement that curriculum recommendations should achieve high coverage of relevant materials.

4.1.3 LightGBM configuration and training objective

All reranking models use LightGBM with a *binary classification* objective and AUC as the optimization metric. Table 2 summarizes the key hyperparameters.

Table 2: LightGBM hyperparameters for the main track ensemble.

Hyperparameter	Value
objective	binary
metric	auc
learning_rate	0.01–0.02
num_boost_round	1600–3200
num_leaves	31
min_data_in_leaf	1024
feature_fraction	0.8
feature_fraction_bynode	0.8
subsample	0.5
subsample_freq	1
is_unbalance	True
monotone_constraints	Yes (on vec_dist)

A notable design choice is the use of monotone constraints on the `vec_dist` feature, which enforces that increasing semantic distance should monotonically decrease the predicted alignment probability. This regularization prevents the model from learning spurious correlations and improves generalization to out-of-distribution topics.

The final ensemble combines predictions from *eight* LightGBM models: four trained on the full feature set (including all TF-IDF, semantic, duplicate, and structural features) and four trained on a reduced feature set (excluding aggregate features like `vec_dist_max` and `match_score_min` to reduce overfitting). Predictions are averaged with equal weight (1/8 each) to produce the final ranking scores.

4.1.4 Complementary IR metrics

While the competition uses F_2^{micro} as the official metric, I also monitor several standard IR metrics to better understand ranking behaviour, particularly at the top of the list:

- **Recall@K.** For topic q , with set of relevant items $\text{Rel}(q)$ and top- K retrieved items $\text{Ret}_K(q)$,

$$\text{Recall@K}(q) = \frac{|\text{Rel}(q) \cap \text{Ret}_K(q)|}{|\text{Ret}_K(q)|}. \quad (5)$$

Recall@50 is particularly informative for checking whether Stage 1 retrieval maintains high recall before reranking.

- **MRR (Mean Reciprocal Rank).** The reciprocal rank of the first relevant item averaged across all topics. This metric is sensitive to the position of the highest-ranked correct answer and is useful for sanity-checking whether the reranker places at least one relevant item near the top.
- **NDCG@10 (Normalized Discounted Cumulative Gain).** A ranking metric that rewards placing highly relevant items at the top of the list, with logarithmic discounting for lower positions. NDCG@10 captures the overall quality of the top-10 ranking better than Precision@10 alone.

These complementary metrics are computed on validation folds but are not used for model selection or leaderboard ranking. Table 3 summarizes the role of each metric.

Table 3: Metrics used in this project.

Metric	Symbol	Used for
Micro F_2 score (official)	F_2^{micro}	Leaderboard, model selection
Recall@K	Recall@K	Retrieval quality (Stage 1)
Precision@K	Precision@K	Top-list quality (Stage 2)
MRR	MRR	First-hit position diagnostic
NDCG@10	NDCG@10	Overall top-10 ranking quality
ROC-AUC	AUC	LightGBM training metric

4.2 Offline Validation Results

I define several progressively stronger model configurations to tackle this problem, ranging from single-backbone models to a heavy multi-model ensemble. Table 4 reports 4-fold GroupKFold cross-validation metrics with a structured layout that separates model architecture from configuration details.

Table 4: Offline 4-fold GroupKFold validation metrics (backbone and ensemble progression).

Backbone	Config name	Emb Dim	#LGB	F_2 (CV)	AUC	MRR	NDCG@10	Recall@50
TF-IDF	TF-IDF-Only	-	-	0.413	0.70	0.58	0.59	0.85
mBERT-base	Single-mBERT	768	4	0.595	0.86	0.79	0.73	0.92
XML-R base	Single-XLMR-Base	768	4	0.601	0.87	0.81	0.75	0.93
XML-R large (2-stage)	Single-XLMR-Large2Stage	1024	4	0.615	0.88	0.84	0.79	0.95
3-Backbone Ensemble	Ensemble-3Backbone-8LGBM	2560	8	0.617	0.89	0.86	0.81	0.96

The experimental evaluation progresses through four distinct configurations. Single-mBERT utilizes bert-base-multilingual-uncased (768-dim) as the sole semantic backbone, establishing a robust multilingual baseline. This is succeeded by Single-XLMR-Base, which leverages xlm-roberta-base (768-dim) to exploit improved pretraining data coverage. To address high-complexity samples, Single-XLMR-Large2Stage deploys a high-capacity xlm-roberta-large (1024-dim) backbone trained via a two-stage curriculum: initially for 20 epochs with a dynamic ArcFace margin ($0.1 \rightarrow 0.5$) and adaptive learning rate, followed by 2 epochs of fine-tuning with a fixed margin. Finally, the Ensemble-3Backbone-8LGBM integrates all three backbones, mBERT-base, XLM-R base, and XLM-R large, via weighted concatenation ($w = [0.3, 0.3, 0.4]$), resulting in a 2560-dimensional embedding space. The

second stage of this ensemble aggregates predictions from 8 LightGBM models (4 full-feature and 4 reduced-feature) to minimize variance.

The validation results highlight a clear correlation between model capacity and retrieval performance. Increasing the backbone complexity from mBERT to XLM-R Base and subsequently to the two-stage XLM-R Large yields consistent improvements across all IR metrics, with F_2 gains of +0.006 and +0.014, respectively. Notably, the significant jump achieved by the Single-XLMR-Large2Stage configuration underscores the efficacy of combining high-parameter models with curriculum learning strategies such as dynamic margin scheduling. The final ensemble achieves the highest performance with an F_2 of 0.617, an AUC approaching 0.89, and peak values for MRR and NDCG@10. This additional +0.002 gain over the single-best backbone demonstrates that fusing diverse architectures and feature subsets effectively captures complementary failure modes. Furthermore, Recall@50 remains consistently high (> 0.92) across all iterations, confirming that the Stage 1 retrieval consistently maintains broad coverage, allowing the stronger models to focus on refining the ranking precision at the top of the list.

4.3 Leaderboard / Test-set Performance

On the held-out test set, I evaluate the same configurations on the main track. The test-set performance closely tracks validation trends, with the ensemble maintaining its advantage. Table 5 summarizes the results.

Table 5: Test-set performance for selected configurations (structured).

Backbone	Config name	Emb Dim	#LGB	F_2 (Public)	F_2 (Private)
mBERT-base	Single-mBERT	768	4	0.593	0.591
XLM-R base	Single-XLMR-Base	768	4	0.599	0.597
XLM-R large (2-stage)	Single-XLMR-Large2Stage	1024	4	0.608	0.606
3-Backbone Ensemble	Ensemble-3Backbone-8LGBM	2560	8	0.617	0.617

The test-set results corroborate the validation findings, exhibiting a monotonic improvement in performance commensurate with model complexity. The progression from Single-mBERT ($F_2 = 0.591$) through the intermediate XLM-R configurations to the final Ensemble-3Backbone-8LGBM ($F_2 = 0.617$) validates the architectural decisions. Moreover, the high consistency between public and private leaderboard scores, differing by no more than ± 0.002 across all models, indicates that the validation protocol is robust and that the solution generalizes well without overfitting to the public test split.

4.4 Ablation and Variant Analysis

To better understand which components contribute most to the final system’s performance, I conduct ablation studies along several axes that directly correspond to the main design choices of the pipeline: candidate pool size in Stage 1, backbone capacity and embedding weights, feature-group selection in Stage 2, ensemble depth, and post-processing rules.

4.4.1 Effect of candidate pool size (Stage 1 retrieval)

I first vary the number of candidates returned by the Stage 1 TF-IDF retriever, while keeping Stage 2 (features, LightGBM, and post-processing) fixed. Table 6 reports the effect on validation metrics.

Table 6: Ablation: candidate pool size in Stage 1 (validation metrics).

Config	#Candidates	F_2 (CV)	Recall@50
S1-Top10	10	0.603	0.930
S1-Top20 (default)	20	0.617	0.960
S1-Top50	50	0.617	0.962

Expanding the candidate pool from 10 to 20 results in substantial improvements for both recall and F_2 (rising from 0.603 to 0.617), aligning well with the F_2 metric’s sensitivity to false negatives. While extending the pool further to 50 candidates yields the highest recall (Recall@50=0.975) but still remains the equal F_2 compared to Top-20 config. Consequently, selecting the Top-20 candidates represents the optimal efficiency-performance trade-off, preserving the high-recall regime essential for the task while maintaining a manageable inference budget.

4.4.2 Backbone capacity and embedding weight ablation

Next, I examine two related questions: (i) how much we gain by increasing backbone capacity, and (ii) how sensitive the system is to the choice of embedding weights in the three-backbone ensemble. Table 7 focuses on single-backbone capacity, while Table 8 studies different weightings of the three-backbone ensemble.

Table 7: Ablation: backbone capacity (single-backbone models, validation metrics).

Backbone	Dimensions	F_2 (CV)	AUC	NDCG@10
Single-mBERT	768	0.595	0.86	0.73
Single-XLMR-Base	768	0.601	0.87	0.75
Single-XLMR-Large2Stage	1024	0.615	0.88	0.79

The backbone ablation analysis reveals a stepwise performance enhancement linked to pretraining quality and model size. Transitioning from Single-mBERT to Single-XLMR-Base yields a modest but consistent gain across all metrics (+0.006 in F_2), reflecting the superior multilingual representation of XLM-R. However, the most significant leap occurs when adopting the XLM-R Large backbone with two-stage training, which delivers a +0.014 increase in F_2 and a substantial +0.04 boost in NDCG@10. This finding suggests that increased parameter count, when coupled with curriculum training, is particularly critical for resolving harder, low-resource topics.

Regarding the fusion weights for the ensemble, completely excluding the large backbone results in the lowest performance, reaffirming the importance of high-capacity models. While equal weighting improves upon the "No-Large" baseline, the default configuration (0.3,0.3,0.4) achieves the optimal F_2 score by slightly favoring the stronger XLM-R Large model while retaining diversity from mBERT and XLM-R Base. Interestingly, an overly aggressive weighting towards the large model (Large-Heavy) marginally improves ranking metrics like AUC and NDCG@10 but causes a slight regression in F_2 . This implies that excessive reliance on a single backbone may lead to over-specialization in certain

Table 8: Ablation: embedding weight configurations in the 3-backbone ensemble (validation metrics).

Config	$(w_{\text{mBERT}}, w_{\text{Base}}, w_{\text{Large}})$	F_2 (CV)	AUC	MRR	NDCG@10
No-Large	(0.5, 0.5, 0.0)	0.608	0.880	0.84	0.790
Equal-Weights	(0.33, 0.33, 0.33)	0.613	0.885	0.85	0.800
Default-Weights (used)	(0.3, 0.3, 0.4)	0.617	0.890	0.86	0.810
Large-Heavy	(0.2, 0.3, 0.5)	0.616	0.892	0.86	0.812

languages or channels, thereby hurting the overall recall-oriented performance required for the F_2 metric.

4.4.3 Effect of feature groups in Stage 2

To quantify the contribution of different feature groups in the Stage 2 LightGBM reranker, I remove groups of features while keeping the rest of the pipeline fixed. The feature groups are: semantic similarity (transformer embeddings), lexical similarity (TF-IDF scores), duplicate/graph signals, and structural/length features. Table 9 presents the results.

Table 9: Ablation: feature-group removal in Stage 2 (validation metrics).

Config	F_2 (CV)	AUC	MRR	NDCG@10	Recall@50
Full-Features (used)	0.617	0.890	0.861	0.810	0.960
No-Semantic	0.588	0.855	0.812	0.760	0.958
No-Lexical	0.606	0.880	0.848	0.796	0.945
No-Duplicate-Graph	0.612	0.885	0.854	0.802	0.957
No-Structural-Length	0.614	0.887	0.857	0.805	0.959

The feature ablation study demonstrates that semantic features are the primary driver of performance; removing them leads to the most drastic drop in F_2 ($0.617 \rightarrow 0.588$) and ranking quality. While Recall@50 remains stable, since candidate retrieval is fixed in Stage 1, the reranker loses the ability to effectively order relevant items without semantic guidance. Lexical TF-IDF features also prove crucial, as their removal degrades performance significantly, particularly for Recall@50, highlighting their role in capturing keyword-specific cues that semantic models might overlook. Duplicate and graph-based features provide a smaller but meaningful boost by transferring structural knowledge from training data to unseen topics, while structural and length features act as beneficial soft priors, offering minor improvements in ranking precision.

4.4.4 Ensemble depth and model diversity

I then isolate the contribution of ensemble depth and model diversity at the LGBM level, starting from a single XLM-R large backbone and gradually increasing the number of models and backbones. The configurations and results are summarized in Table 10.

The breakdown of ensemble gains illustrates the additive value of variance reduction and representation diversity. Expanding from a single LGBM model to a 4-fold ensemble improves F_2 by +0.004 simply by averaging out data split noise. Introducing backbone diversity by combining mBERT

Table 10: Ablation: ensemble depth and backbone diversity (validation metrics).

Ensemble configuration	Backbones	#LGBM	F_2 (CV)	AUC	NDCG@10
Single-Large-1LGBM	XLM-R large only	1	0.608	0.880	0.780
Single-Large-4LGBM	XLM-R large only	4	0.612	0.885	0.795
TwoBackbone-4LGBM	mBERT + XLM-R large	4	0.615	0.887	0.807
ThreeBackbone-8LGBM (used)	mBERT + XLM-R base + XLM-R large	8	0.617	0.890	0.810

with XLM-R Large further lifts performance to 0.615, as the distinct pretraining objectives capture complementary linguistic patterns. The final configuration, utilizing three backbones and eight LGBM models (varying feature subsets), achieves the peak F_2 of 0.617. This confirms that simultaneously diversifying feature subsets and backbone architectures acts as a robust regularizer, preventing overfitting to any single representation.

4.4.5 Post-processing and thresholding

Finally, I analyze the impact of the post-processing rules that convert LGBM scores into the final set of recommended content items for each topic. These rules include (i) a top-1 guarantee per topic, (ii) relaxed thresholds for content seen during training, and (iii) content-centric constraints for unseen content. Table 11 compares several variants.

Table 11: Ablation: post-processing and thresholding (validation metrics).

Config	F_2 (CV)	Precision	Recall	MRR	NDCG@10
NoPost (fixed top-10)	0.607	0.520	0.785	0.858	0.804
Post-Top1Only	0.612	0.510	0.810	0.860	0.806
Post-Default (used)	0.617	0.500	0.840	0.861	0.810
Post-Strict (aggressive filter)	0.610	0.545	0.760	0.859	0.808

The post-processing ablation underscores the necessity of aligning model outputs with the specific requirements of the competition metric. Relying solely on a fixed top-10 cutoff (NoPost) yields decent precision but suboptimal recall, capping the F_2 score at 0.607. The default post-processing strategy strategically trades a small amount of precision ($0.520 \rightarrow 0.500$) for a substantial gain in recall ($0.785 \rightarrow 0.840$), boosting F_2 to 0.617. This behavior perfectly targets the micro- F_2 objective, which weights recall four times more heavily than precision. Conversely, applying overly aggressive filtering increases precision but penalizes recall too severely, resulting in a net performance loss.

5. Conclusion

This project tackled the problem of aligning open educational resources with curriculum topics at scale. The proposed solution combines a character 4-gram TF-IDF retriever for high-recall candidate generation with a feature-rich LightGBM reranker driven by multilingual transformer embeddings and structured signals derived from the training graph. Careful feature engineering, multilingual ensembling, and targeted post-processing yield strong performance on the official micro-averaged F_2 metric while keeping inference efficient through offline embedding precomputation and a small candidate set per topic.

Beyond its competition results, the system illustrates how classical Web Mining techniques and modern NLP models can be integrated into a practical curriculum alignment pipeline. The same architecture – fast lexical retrieval followed by a learned semantic reranker – is broadly applicable

to other matching tasks such as mapping documents to taxonomy nodes or pairing questions with candidate answers. More generally, the work highlights the value of hybrid systems that combine interpretable IR heuristics with flexible learned components. With further work on hierarchy-aware modelling, feedback integration, and distillation, this line of research can support educational platforms in delivering more targeted, equitable access to learning resources at globalscale.

Acknowledgements

We would like to express my sincere gratitude to the Doctor Nguyen Kiem Hieu for providing the theoretical foundation and practical guidance that made this project possible. Their lectures on Information Retrieval, Recommendation Systems, and Opinion Mining directly inspired the design of the two-stage ensemble reranking pipeline presented in this report.

Team Members and Contributions

No.	Full Name	Student ID	Contribution	Main Tasks
1	Nguyen Hai Dang	20225478	20%	mBERT, LightGBM
2	Nguyen Vo Ngoc Khue	20225504	20%	RoBERTa-Base
3	Nguyen Hoang Quan	20225519	20%	RoBERTa-Large
4	Nguyen Tien Dat	20225481	20%	TF-IDF, Ensemble Pipeline
5	Phan Minh Hoa	20225495	20%	GUI, Docs

References

1. Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
2. Liu, T. Y. (2009). "Learning to Rank for Information Retrieval." Foundations and Trends in Information Retrieval, 3(3), 225-331.
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." arXiv preprint arXiv:1810.04805.
4. Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." EMNLP 2019.
5. Conneau, A., et al. (2020). "Unsupervised Cross-lingual Representation Learning at Scale." ACL 2020.
6. Chen, T., & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." KDD 2016.
7. Ke, G., et al. (2017). "LightGBM: A Fast, Distributed, High-Performance Gradient Boosting Framework." NeurIPS 2017.
8. Järvelin, K., & Kekäläinen, J. (2002). "Cumulated Gain-Based Evaluation of IR Techniques." ACM TOIS.
9. Learning Equality Curriculum Recommendations Competition. Kaggle, 2023.